

Documento explicativo dos codigos usados no projeto

Este documento explica as estruturas de controle usadas nos scripts do jogo em Unity/C#: if, else, else if, for e while. A explicação está ligada aos scripts enviados: PlayerMovement.cs, PlayerHydration.cs, PlantWaterUI.cs, PlantInteraction.cs, GameManager.cs e PlantGrowth.cs.

1. O que são estruturas de controle

Estruturas de controle servem para controlar o caminho que o programa vai seguir.

No jogo, elas são usadas para tomar decisões, como:

- verificar se o jogador está no chão;
- verificar se a planta recebeu água suficiente;
- verificar se o jogador está perto da planta;
- verificar se a vida/hidratação chegou a zero;
- repetir uma ação para todos os modelos da planta.

2. Uso do if

O if significa "se". Ele executa um bloco de código apenas quando uma condição é verdadeira.

Formato básico:

```
if (condição)
{
    // Código executado se a condição for verdadeira
}
```

Exemplo em PlayerMovement.cs

```

if (controller.isGrounded)
{
    moveDirection = (forward * vertical + right * horizontal) * moveSpeed;
    moveDirection.y = -1f;
}
else
{
    moveDirection.y -= 20f * Time.deltaTime;
}

```

Esse trecho verifica se o personagem está no chão com `controller.isGrounded`. Se estiver no chão, o jogador pode se mover normalmente. O código calcula a direção do movimento usando as teclas WASD e aplica a velocidade.

Se não estiver no chão, o jogador está no ar. Então o `else` aplica uma força para baixo, simulando a gravidade.

Exemplo em `PlantWaterUI.cs`

```

if (plant != null)
{
    plant.ReceiveWater(amount);
    GameManager.Instance.RegisterWater(amount);
}

```

Esse `if` verifica se a variável `plant` realmente possui uma planta salva nela. Isso evita erro no jogo. Se `plant` for `null`, significa que não existe uma referência válida para a planta, então o código dentro do `if` não roda.

Exemplo em `PlantInteraction.cs`

```

if (other.CompareTag("Player"))
{

```

```
waterUI.ShowUI(GetComponent<PlantGrowth>());  
}
```

Esse if verifica se o objeto que entrou na área da planta e o jogador. Se for o jogador, a interface de dar água aparece.

No OnTriggerExit, a lógica é parecida:

```
if (other.CompareTag("Player"))  
{  
    waterUI.HideUI();  
}
```

Quando o jogador sai da área da planta, a interface desaparece.

3. Uso do else

O else significa "senão". Ele é executado quando a condição do if é falsa.

Formato básico:

```
if (condição)  
{  
    // Executa se for verdadeiro  
}  
else  
{  
    // Executa se for falso  
}
```

Exemplo em PlayerMovement.cs

No movimento do jogador, o if trata o caso em que o personagem está no chão. O else trata o caso em que ele não está no chão, aplicando a gravidade.

Exemplo em PlantGrowth.cs

```
if (conditionsMet)
{
    growthStage = Mathf.Min(growthStage + 1, 2);
    UpdateVisual();

    Debug.Log("Planta cresceu! Estagio: " + growthStage);
}
else
{
    if (growthStage > 0) growthStage--;
    UpdateVisual();

    Debug.Log("Planta não cresceu");

    if (growthStage <= -1)
    {
        isDead = true;

        Debug.Log("A planta morreu...");
    }
}
```

Aqui o if verifica se todas as condições para a planta crescer foram cumpridas:

```
bool conditionsMet = hasNutrients && lightOn && waterToday >= WATER_NEEDED;
```

Isso quer dizer que a planta só cresce se recebeu nutrientes, se a luz está ligada e se recebeu água suficiente.

Se tudo isso for verdadeiro, a planta cresce. Se alguma condição for falsa, entra no else: a planta não cresce, pode perder estagio e pode morrer.

4. Uso do else if

O else if é usado quando existem várias possibilidades diferentes.

Formato básico:

```
if (condicao1)
{
    // Caso 1
}
else if (condicao2)
{
    // Caso 2
}
else
{
    // Caso nenhuma condição anterior seja verdadeira
}
```

Exemplo em PlayerHydration.cs

```
if (waterGivenToPlant >= 2)
{
    hydrationDays--;
}
else if (waterGivenToPlant == 1)
{
    Debug.Log("Deu 1 caneca -> hidratacao estavel");
}
else
{
    if (hydrationDays < 5)
    {
```

```
hydrationDays++;  
}  
}
```

Esse trecho controla a hidratação da criança.

- Se a criança deu 2 ou mais canecas para a planta, ela perde 1 dia de hidratação.
- Senão, se ela deu exatamente 1 caneca, a hidratação fica estável.
- Senão, significa que ela deu 0 canecas. Nesse caso, ela pode recuperar hidratação, desde que ainda esteja abaixo de 5.

Esse é um bom uso de else if, porque existem três situações possíveis: deu muita água para a planta, deu uma quantidade equilibrada ou não deu água.

Exemplo com cores da hidratação

```
if (hydrationDays <= 2)  
    hydrationText.color = Color.red;  
else if (hydrationDays <= 3)  
    hydrationText.color = Color.yellow;  
else  
    hydrationText.color = Color.green;
```

Esse trecho muda a cor do texto dependendo da situação: vermelho quando a hidratação está muito baixa, amarelo quando está em alerta e verde quando está boa.

5. if dentro de outro if

Também foi usado um if dentro de outro if. Isso é chamado de condição aninhada.

Exemplo em PlayerHydration.cs

```
if (hydrationText != null)
{
    hydrationText.text = "Hidratacao: " + hydrationDays + " dias";
```

```
    if (hydrationDays <= 2)
        hydrationText.color = Color.red;
    else if (hydrationDays <= 3)
        hydrationText.color = Color.yellow;
    else
        hydrationText.color = Color.green;
}
```

Primeiro, o código verifica se existe um texto de hidratação na tela. Só depois disso ele altera o texto e a cor. Isso evita erro, porque se hydrationText fosse null, o Unity não conseguiria alterar text nem color.

6. Uso do operador ternário

No PlayerMovement.cs, existe uma forma curta de if e else, chamada operador ternário:

```
float moveSpeed = Input.GetKey(KeyCode.LeftShift) ? runSpeed : speed;
```

Isso funciona assim:

```
condição ? valor_se_verdadeiro : valor_se_falso;
```

No código, se o jogador estiver segurando LeftShift, a velocidade sera runSpeed. Se não estiver segurando LeftShift, a velocidade será speed.

E como escrever:

```
if (Input.GetKey(KeyCode.LeftShift))
{
    moveSpeed = runSpeed;
```

```
}  
else  
{  
    moveSpeed = speed;  
}
```

7. Uso do for

O for é usado para repetir uma ação várias vezes. Ele é muito comum quando precisamos percorrer uma lista ou array.

Formato básico:

```
for (início; condição; incremento)  
{  
    // Código repetido  
}
```

Exemplo em PlantGrowth.cs

```
for (int i = 0; i < stageModels.Length; i++)  
{  
    if (stageModels[i] != null)  
        stageModels[i].SetActive(i == growthStage);  
}
```

Esse for percorre todos os modelos visuais da planta.

- `int i = 0`: começa no índice 0, ou seja, no primeiro modelo.
- `i < stageModels.Length`: continua repetindo enquanto `i` for menor que a quantidade de modelos.
- `i++`: aumenta o valor de `i` em 1 a cada repetição.

Dentro do for, o código verifica se aquele modelo existe:

```
if (stageModels[i] != null)
```

Depois, executa:

```
stageModels[i].SetActive(i == growthStage);
```

Esse comando ativa apenas o modelo que corresponde ao estágio atual da planta. Se growthStage for 0, ativa o modelo 0. Se for 1, ativa o modelo 1. Se for 2, ativa o modelo 2. Os outros modelos ficam desativados.

8. Uso do while

No projeto não existe nenhum while.

Mesmo assim, o while funciona assim: ele repete um bloco de código enquanto uma condição for verdadeira.

Formato básico:

```
while (condição)
{
    // Código repetido enquanto a condição for verdadeira
}
```

Exemplo simples:

```
while (energia > 0)
{
    energia--;
}
```

Esse código continuaria rodando enquanto energia fosse maior que 0.

No Unity, é preciso tomar cuidado com while, porque se a condição nunca ficar falsa, o jogo pode travar. Por isso, no projeto, o for foi uma escolha melhor para percorrer os modelos da planta, já que o número de repetições é conhecido.

9. Resumo por script

PlayerMovement.cs

Usa if e else para verificar se o personagem está no chão. Também usa operador ternário para escolher entre velocidade normal e velocidade de corrida.

PlayerHydration.cs

Usa if, else if e else para decidir o que acontece com a hidratação da criança dependendo da quantidade de água dada para a planta. Também usa condições para mudar a cor do texto da hidratação.

PlantWaterUI.cs

Usa if para verificar se a planta existe antes de enviar água para ela. Também usa if para fechar a interface quando o jogador está perto da planta e aperta Escape.

PlantInteraction.cs

Usa if para verificar se quem entrou ou saiu da área da planta foi o jogador.

GameManager.cs

Usa if para verificar se o jogo acabou:

```
if (playerHydration.hydrationDays <= 0 || plant.isDead)
{
    Debug.Log("=== FIM DO JOGO ===");
}
```

Esse if usa ||, que significa "ou". Ou seja, o jogo acaba se a hidratação da criança chegar a 0 ou se a planta morrer.

PlantGrowth.cs

Usa if e else para decidir se a planta cresce ou não. Usa for para passar por todos os modelos da planta e ativar apenas o modelo do estágio atual.

10. Conclusão

No projeto, as estruturas condicionais foram usadas principalmente para tomar decisões importantes do jogo.

O if verifica uma condição. O else executa uma alternativa quando o if é falso. O else if permite testar várias possibilidades. O for repete uma ação para vários elementos, como os modelos da planta. O while não foi usado no projeto, mas serve para repetir uma ação enquanto uma condição continuar verdadeira.

Essas estruturas são essenciais porque fazem o jogo reagir às ações do jogador e ao estado dos objetos, como água, crescimento da planta, hidratação e fim de jogo.